Maintain role even on refusal (P22)	*“maintain its role-playing ability even when refusing to answer” (P22)*

Retain Role-Specific Perspective

Action	Examples & Quotes
Keep domain lens for diverse theme formulation (P27)	*“others may retain role-specific perspectives to support diverse theme formulation” (P27)*

V3 (validated):

- Memory Actions

Action	Plain Explanation	Evidence
Store information in working memory	Put whatever the agent is currently using — recent inputs, intermediate results — onto a "scratch pad" for quick access.	
Store/update experiences in episodic memory	Save past events as "episodes" — which game was won, which plan failed, what happened step by step — like a personal diary.	

Store knowledge in semantic memory	Save general facts about the world — "fire is hot", "404 means not found" — not tied to any specific event.	
Discard information from working memory		"the local memory is discarded upon successful completion of the subtask" P1
Consolidate working memory into long-term memory		This discard-after-use strategy ensures that only the clear and successful analysis path is retained in the global memory, preventing interference from the intermediate trial-and-error history and thereby maintaining a concise and efficient context for subsequent steps. P1
TODO: Feel like we should have long-term memory, anyone has found evidence for long-term memory?		

- Grounding Actions

Action	Plain Explanation	Evidence
Process perceptual inputs into textual observations	Turn images, sounds, or sensor readings into text descriptions so a text-based LLM can understand the physical world.	
Affect physical environments via robotic planners	Actually move things in the real world — pick up a cup, open a door — by sending language commands to a robot arm.	
Accept instructions from humans	Receive a task or command from a human user, e.g. "book me a flight" or "translate this paragraph."	

Ask for clarification from people	When something is unclear, proactively ask the human — "Which file did you mean?"	
Entertain people	Chat casually, tell jokes, do role-play — provide emotional support or fun.	“Emotional expressivity remains largely limited in current S2S systems” (P11) “models exhibit a strong default tendency to excessively affirm, apologize, and express gratitude” (P11)
Conduct multi-agent debate	Have multiple agents argue different sides of the same question, using the debate to reach a better answer.	
	Communicate via Structured Dialogue	*"LLM-driven agents communicate via structured chat chains across design, coding, and testing phases" (P28)*
Solve tasks collaboratively across agents	Split a big job among multiple agents — one writes code, one tests, one documents — and combine the results. Communicate with peer agents	*"A central orchestrator maintains a high-level route plan while dynamically dispatching sub-agents based on game context" (P25)* *Upon successful generation of a result, the Evaluator agent, ALLMe assesses the outcome and provides natural language feedback if it deems revisions are necessary. P1*
	Decompose Development Roles	*"assembling a team of three LLM roles — analyst, coder, and tester — responsible for analysis, coding, and testing" P28*
~~Communicate with peer agents~~		
Visualize results for human consumption	**Format structured output for verification**	*CellAgent generated differential expression and marker gene visualizations,* *enabling intuitive interpretation of cluster identities and potential biological differences (Fig. 13).* P1 *"generate outputs in a JSON format (or lists of JSON objects)" (P2)*

Interact with games, APIs, and websites	Operate in digital environments: play Minecraft, call a weather API, click buttons on a webpage. Navigate and interact with web interfaces Invoke structured API or data endpoint Interacting with physical environment Interacting with digital environment	*“navigate multiple websites, extract information from both structured and unstructured sources" P2* *"Game Control: get_game_state, press_buttons, and navigate_to for direct game control"(P25)*
	Analyse source code	*"analyzes the project's core functionalities and their interactions with UI elements by reading the source code" (P28)*
	Annotate UI Components	*"annotate_interactive_components(file, strategy='add data-testid')" P28*
Execute code	Run a piece of code and get the output — e.g. run a Python script, execute a database query.	*Test Runner executes the script to detect logical inconsistencies or failed assertions. (P28)*
Augment external knowledge	Call outside tools to fill knowledge gaps — look up a database, etc.	
	Generate Buggy Code Sample	*"LLM which generates buggy code by integrating retrieved error slices from BugRAG as context" (P29)*
Augment external computation	Call outside tools to handle heavy math, use a calculator, etc.	
~~Simulate human persona~~	~~Initialize a personalized agent by combining user demographic attributes (e.g. gender, region, personality tag) and historical posts into a prompt template that shapes the agent's behavioral perspective then simulate behaviors without actual executing them~~	~~*"global interactive multi-behavior overlapping network is constructed based on the simulated behaviors of reposting and reposting with a comment, as well as following" (P21) “construct prompt templates for creating reader agents based on user attributes ua and user historical posts” (P21) "I want you to play as {role}, imitating {role}'s personality and values" (P22) "maintain its role-playing ability even when refusing to answer"(P22)*~~

Inform user of task outcome	Proactively communicate task completion status or results to the human user without being directly asked — e.g. 'Your reservation has been updated.'	*Agent should inform the user: 'Buy a nice rich navy bathing dress (P3)* *Agent should inform the user that wifi is turned off (P3)*
Produce final answer	Produce the final response after the reasoning chain has been grounded in visual or contextual evidence; commit to a definitive answer and terminate the reasoning trajectory. Terminate the process	*"Final Answer: The average number of lines per scene in the ep_7.csv dataset is approximately 6.02" (p32)* *"produces a final answer and terminates the rollout; enclose it within <answer></answer> tags" (P4)*
Analyze codebase structure and behavior		*"analyzes the project's core functionalities and their interactions with UI elements by reading the source code" P28* *"Analyze frontend framework and interactive components" P28*
~~Collect observation from environment~~		~~*"providing a noisy observation x ~ y(s) = f(s) + nu(s) at state s"*~~
~~Generate analysis code~~	~~Write Python code to perform a specified data analysis task — distinct from executing the code. The agent authors code based on the task plan, then submits it for execution.~~	~~*The process includes reasoning, code writing, execution, and iterative refinement based on the results obtained*~~
~~Engineer new data feature~~	~~Create a new derived variable from existing data columns — combining, transforming, or encoding columns to expose analytical signal for downstream computation.~~	~~*"Create a new feature called FamilySize by summing the SibSp and Parch columns" P32*~~
Grounding failure		with common failures in temporal and cross-video grounding," (P13)

- Retrieval Actions

Action	Plain Explanation	Evidence
Load skills from the skill library	Find and load a ready-made code snippet from a pre-built library — e.g., grab the "chop tree" skill in Minecraft.	
Retrieve events from episodic memory	Search the "diary" for relevant past experiences, ranked by how recent, important, and relevant they are.	*"Persistent memory system storing discoveries (locations, NPCs, items, strategies) with importance-weighted retrieval" (P25)*
	Retrieve Analogical Examples	*"Recall three (03) relevant and distinct problems (different from the user task)" P28*
Leverage documents for code generation	Look up reference docs (API docs, tutorials) and use that information to help write code.	explicit retrieval can interfere with the immediate rule execution required by procedural tasks" (P12)
Retrieve knowledge from semantic memory	Pull a general fact from the knowledge base — e.g., look up "what does HTTP 404 mean?"	
Extract data from structured and unstructured documents	Read reference documents (papers, READMEs, API docs, code files) and extract relevant information to support the current task — not limited to code generation; purpose may be comprehension, inspection, or analysis.	*"diverse filetype reading; read between 1 to 15 documents and/or tables" P2* *"read the README file — Phase 1: read_file('reproduction_package/readme.txt')" (P11)*
Retrieve Domain Context		*"LLM leverages retrieved HLS-related context to transform input C algorithms or natural language descriptions" (P29)*
Query Error Repository		*"queries BugRAG to check for existing entries" P29*

- Reasoning Actions

Action	Plain Explanation	Evidence
Read from working memory	Check the "scratch pad" — grab the intermediate results and state information stored there.	“IMPLICITMEMBENCH reframes evaluation from ‘what agents recall’ to ‘what they automatically enact’.” (P12)
Summarize recent observations, trajectories	Condense a flood of recent information into a few key takeaways to avoid information overload.	“Models memorize individual rules but fail to integrate them” (P12) “First, models are primarily confused by cross-video and temporal distractors, while scene distractors are the easiest, indicating that models handle static visual input better than temporal and cross-video reasoning.” (p14)
Distill insights from retrieved information	Extract the core conclusion from a pile of raw data.	*For every binary score z(q) i,j from the judge, there is a corresponding explanation e(q)i,j.* P3
	Generate initial codes	*"Each agent then emits a set of initial codes; cluster semantically similar codes and generate preliminary themes" P27*
	Generate requirements	*"automatically generates candidate user-facing requirements based on this analysis" P28*
Provide context for subsequent LLM calls	Package the reasoning results so far and feed them as background into the next LLM call for a better answer.	
	Maintain Cross-File Consistency	*"Agents share project context to ensure: Cross-file consistency; Non-conflicting test-ids" P28*

Apply a constraint or domain rule		*"codified best-practices, such as the standard order of operations (e.g., quality control must precede normalization)" P1*
	Apply previously learned rules	*“Tool & API Usage* *Override ingrained calling conventions* *Reversed Parameters (dst→src), Session Prefix (auth fusion), Alien Filesystem (custom separators)* *Linguistic Formats* *Internalize arbitrary templates* *Scribe’s Signature (fixed wrapper), Corporate Etiquette (mandated greeting/closing)* *Logical Operations* *Apply non-standard operators* *Omega: a Ω b = 2a + b², Modified Fibonacci: F(n) = F(n−1) + 2F(n−2)* *Abstract Rules* *Form habits in micro-worlds* *Forbidden Square (board constraint), Triple Knock (ritual sequence)* *Creative Constraints* *Maintain style without reminders* *Voice Consistency (no first-person), Botanical Similes (nature metaphors)” (P13)*
Synthesize results across subtasks		*"the final results from all subtasks are synthesized to meet the user's requirements" P1*

Decompose into atomic knowledge units	Break a structured description (e.g., a character profile, a document) into discrete, independently verifiable atomic facts that can be queried or evaluated one at a time.	*"break down the given character description into multiple atomic pieces of knowledge" (P22)*
Compact context window	Selectively prune the agent's conversation history to stay within the LLM's token budget, discarding redundant game state while preserving key decisions. Distinct from Summarize: goal is budget management, not insight extraction.	*"automatic context compaction to manage the thousands of reasoning steps required, preserving only LLM responses and action summaries while discarding redundant game state" (P25)*
Infer hidden state from observable evidence	Abductively reconstruct latent or opponent-private information by reasoning backward from indirect observations (e.g. damage dealt, move ordering, item effects reveal hidden stats and equipment).	*"intelligently infers hidden information through game mechanics: damage calculations reveal stat distributions, move priority ordering constrains speed ranges" (P25)*
Infer structure from environmental artifacts	Construct an implicit model of an unknown environment (directory layout, data organization, file relationships) by reading and reasoning across available clues such as README files and package structures.	*"agents must infer the directory layout by inspecting package structures and README files" (P11)*
Extract action label	Identify and assign the underlying strategic or communicative action type to each turn in a raw interaction transcript.	*"METRO starts with the identification of the strategic action a_i for every expert utterance u_i in transcript D" (P10)*

Apply scientific reasoning to evidence	Apply logical, mathematical, visual, or causal reasoning to analyze external evidence (paper findings, code outputs, data points) and reach an analytic conclusion — distinct from condensing internal memory.	*"logical reasoning to interpret papers and code; mathematical reasoning to modify and run code; causal reasoning to infer scientific insights from results" (P11)*
Correct reasoning direction mid-trajectory	Within an ongoing reasoning chain, update the working hypothesis and redirect the inference path based on new evidence gathered in the current trajectory — without waiting for episode completion.	*"allowing the model to update its textual understanding, refine its hypothesis, or even trigger further visual exploration" (P4)*
Sample diverse reasoning trajectories	Generate multiple parallel full-trajectory rollouts from a shared context point, exploring different reasoning/action paths simultaneously to cover a range of hypotheses before aggregating or selecting.	*"Fork the current generative state and generate a new parallel trajectory from this point of high uncertainty" P4*
Apply Analogical Reasoning		*"generates solutions based on analogies to unrelated projects, which resemble few-shot prompting" P28*
Diagnose Error Cause		*The analysis LLM 3 then examines the error causes and provides debugging instructions, as illustrated in Table 1. P29*
Compare and analyse		*The LLM compares the QoR before and after optimization. It observes that the latency decreases significantly, while the resource utilization (specifically DSP and LUTs) increases. Based on this contrast, the model generates a reasoning step explicitly linking the directive to its hardware impact: "ApplyingUNROLL increases parallelism, which reduces total execution time but requires more logic resources to instantiate parallel hardware units." P29*

- Learning Actions

Action	Plain Explanation	Evidence
Store episodic trajectories	Save a full action sequence from start to finish — every step recorded — for later training or review.	
Update parametric policy	Change the model's internal weights through training so it actually performs better on similar tasks — real learning.	
~~Establish a non-parametric policy~~	~~Instead of changing weights, build a case library of past experiences and look up the closest match when a new problem arrives.~~	
Update semantic memory with knowledge	Generalize from raw experiences into reusable facts and save them — e.g., after three failures, note "this API has a rate limit." **Expand Error Repository**	*"the inspection agent identifies a new error type and integrates the slice into the error repository with a new mnemonic identifier" P29*
Build semantic map via VLM	Use a Vision-Language Model to scan the environment and produce a labeled map — not just layout, but annotations like "door here" or "enemy there."	
Update LLM parameters via SL/RL/RLHF	Adjust the large language model's weights using supervised learning, reinforcement learning, or human feedback to improve on a specific task.	
~~Self-improve~~	~~Have the LLM produce multiple outputs, pick the best ones, and train on those — the model bootstraps its own improvement.~~	
~~Utilize subtask-specific LLMs~~	~~Use smaller, specialized models for specific jobs — one for summarization, one for SQL — instead of one big model for everything.~~	
Update the source code as procedural memory	Modify the agent's own source code — essentially patching itself to change how it behaves.	

Infer instructions from input-output examples	Look at a few "input → output" examples, figure out the underlying rule, and use that rule as a prompt for future tasks.	
Update reasoning via prompt update	Rewrite its own prompt template so the LLM thinks more clearly next time — learning a better way to reason.	
Update grounding via code-based skills	Write or improve code snippets that interact with the outside world — e.g., learn a new way to navigate a webpage.	
Maintain curriculum library	Keep a syllabus of mastered and upcoming skills, arranged by difficulty, so the agent learns in a sensible order.	
Update retrieval procedures	Improve how the agent searches for information — e.g., better keyword strategies or smarter relevance ranking.	
Update decision-making procedures	Improve the decision process itself — e.g., go from "pick randomly" to "score every option and pick the best."	
Self-Correct Step Implementation		*"Modify step implementation based on error message" 28*

- Planning actions

Action	Plain Explanation	Evidence
Decompose a task into subtasks	Break a long-horizon objective into an ordered sequence of subtasks, each with a verifiable completion condition.	*“Decomposes complex user requests into manageable subtasks, an Executor that carries out the analysis by generating and running code, and an Evaluator that assesses the quality of the results” P1* *"formulate plans, decompose problems into sub-steps" P2*

		"LLM decomposes the task into a sequence of subgoals, each paired with an executable success-condition function" (P25) *"agents start by developing a broad understanding of the task and the environment in Phase 1" (P11)* *"I will need to 1. Load the CSV 2. Analyze the dataset 3. Count how many lines 4. Compute the average" (P32)*
	Decompose Task into Subproblems	*"Create JavaScript functionality handling empty display and consecutive operator clicks" (P28)*
Formulate a workflow from task structure	**Formulate multi-step navigation plan**	*"the Planner accurately interprets user intent and formulates a comprehensive analysis workflow" P1* *"planning to navigate the web; tasks require navigation through an average of 4.2 web pages"* *“"I need to load the dataset, conduct a linear regression analysis between GDP per capita and life expectancy" P32*
		"summarizing it into a high-level planning directive that emphasizes cumulative temporal effects" (P8) *"HLSTuner formulates a detailed plan that specifies: (1) the combination of HLS directives, (2) target code segments, and (3) the insertion actions" (P29)*
Plan multi-turn dialogue strategy	Produce a full-task plan, conditioned on the current dialogue state, adapting it to the current task instance.	*"captures planning logic at two distinct temporal resolutions: immediate child nodes represent short-term tactical responses...full branches encapsulate long-term strategic foresight" (P10)* *“following a structured template built upon successful reproducibility assessment cases to improve planning" (P11)*
Generate short-term action suggestion	Reinterpret retrieved breadth-logic actions into a concise, context-specific next-step strategy suited to the current dialogue turn.	*"the LLM reinterprets the actions from breadth logic...summarizing them into a concise next-step strategy"*

Apply fallback strategy	Generate a contingency output (e.g., a dummy result or safe default) before attempting the full plan, ensuring a valid result is available even if later phases fail.	*"incorporating a dummy score prediction as a fallback mechanism" (P11)*
Plan Code Structure **Or Plan tool using?**		*"Planning: HTML Structure, CSS Styling, JavaScript Functionality" (P28)* *"I can use the pandas method corr() to calculate the Pearson correlation coefficient" (P32)*
Assign Agent Roles		*"assigns diverse roles to multiple agents to efficiently decompose complex tasks" (P28)*
Select Directive Strategy		*"selects effective HLS directive combination strategies and inserts directives within the specific structure (e.g., loops and arrays)" (P29)*
~~**Align with requirements**~~		~~*To navigate the expansive design space of HLS C, HLSTuner enables QoR-aware reasoning to align optimization goals with hardware constraints (P29)*~~

• Reflection actions

Action	Plain Explanation/Subcategories	Evidence
Reflect on failures	Do a post-mortem on what went wrong, store the lessons learned, and avoid making the same mistake again. Analyze the current episode's stuck or failed state — comparing observations against ground truth — to identify root causes and trigger immediate in-run recovery.	*"Analyzes stuck states by comparing current situation against ground truth sources (porymap data, knowledge base) to diagnose navigation failures" (P25)*

	Debug Test Logic	*Modify test script based on failure log. P28*
	Align Requirements to Tests	*"Refine requirement based on validated test cases to ensure alignment" P28*
	Self-Correct Step Implementation	*"Modify step implementation based on error message" P28*
Correct errors with self-reflection		*“In case of an execution error E(ci), the Executor autonomously performs self-correction to produce a valid code version. P1*
Adjust action parameters based on feedback		*automatically adjusts parameters using exception information to generate executable code P1*
Iterate toward a quality threshold		*"the optimization process runs three iterations, each invoking a different algorithm" P1*
Incorporate feedback from another agent		*Upon successful generation of a result, the Evaluator agent, ALLMe assesses the outcome and provides natural language feedback if it deems revisions are necessary. P1*
Refine outcome over multiple rounds through self-reflective mechanism		*The Evaluator drives a self-reflective optimization mechanism, which leverages automated evaluation methods for various analysis tasks to iteratively refine outcomes, replacing subjective manual assessments. P1*
~~Learn from mistakes in working memory~~		~~*This allows the agent to learn from mistakes in realtime and avoid repeating errors before the local memory is discarded upon successful completion of the subtask.*~~
~~Self-reflect optimization~~		~~*This self-reflective optimization loop iterates to enhance precision, and the final results from all subtasks are synthesized to meet the user’s requirements.*~~

Diagnose and recover from in-episode error	During a task, detect that a step has failed — due to navigation error, technical failure, or synthesis error — and attempt recovery within the same episode without persisting any lesson for future tasks.	*"Navigation errors — when the agent fails to locate or access the correct source of information; Synthesis Error — when the agent reaches an incorrect conclusion despite accessing the correct information" P2*
Inspect Error Pattern		*an inspection agent examines the erroneous code and error messages parsed from the HLS tool test results. P29*
Reflect on Error Fix		*reflects on the code after assuming the fix to ensure the modification is reasonable P29*

• Tool Use actions

Action	Explanation/Subaction	Evidence
Invoke visual inspection tool	Call an image-manipulation tool (e.g., zoom, crop, windowing) to examine a specific region of the visual input in detail as a targeted mid-reasoning grounding step.	*"proactively invokes the Zoom-in tool for a targeted examination of the specific region of interest" P4*

• Synthesis actions

Action	**Plain Explanation**	**Evidence**

Synthesize information from multiple sources		*"combine information from multiple sources...to produce a coherent solution" P2*
Detect trend in data		*"Identifying patterns, directions, or changes in data over time or across contexts" P2*
Correlate variables across sources		*"Measuring relationships or associations between two or more variables" P2*
Rank items by criteria		*"Ordering items or facts based on specific criteria or importance" P2*
Count and compare values		*"Quantifying occurrences and comparing values across sources or categories"*
Filter information by threshold		*"Selecting relevant information based on criteria, quality, or thresholds" P2*
Compute aggregate value		*"Determining a representative value that summarises numerical data collected from multiple sources" P2*
Infer causal relationship		*"determine the most plausible event or factor accounting for this variability" P2*
Synthesis error		*Synthesis error: When the agent reaches an incorrect conclusion despite accessing the correct information, due to flaws in logical reasoning, interpretation, or multi-step analytical processes. P2* ~~*First, models are primarily confused by cross-video and temporal distractors, while scene distractors are the easiest, indicating that models handle static visual input better than temporal and cross-video reasoning (P14)*~~

- Evaluate actions

Action	Subcategory	
Evaluate actions	**Evaluate actions via LLM-as-judge/heuristics;**	*"violates specific content within the role profile" (P22), "we use GPT-4o as the default evaluator, each dimension scored on a scale of 0 to 2"(P22)*

	Evaluate actions via quantitative metrics; **Evaluate actions via learned value function; Detect query-role conflict**	*"In Phase 2, they inspect the provided code for potential inconsistencies; generating a reproducibility score on a scale from 1 to 4" (P11)*
	Score Annotation Quality	*The Gold Checker evaluates each annotation on three binary criteria: equivalence,completeness, and correctness, where a score of 1 indicates the criterion is met and 0 otherwise. (P28)*
Evaluate output quality via LLM judge		*"Evaluation Scores: Obtain s^(t) = (C, D, T) on the trustworthiness dimensions" P27*
	Score Repair Candidates	*These candidates are then compiled and evaluated by the scoring agent 6 (functioning as the judge). The scoring agent (functioning as the judge). Based on clarity, logical soundness, alignment with error messages, this agent selects the optimal suggestion P29*
	Analyze Quality-of-Results	*"HLSTuner analyzes QoR to scale loop parallelism up or down. For example, if hardware utilization exceeds the budget, HLSTuner halves parallelism" (P29)*
Verify output accuracy via consistency check		*task an evaluator agent with determining whether each theme is consistent with its supporting quotes. P27*
Verify intermediate results		*"agents with domain expertise verify intermediate results and reduce errors" P28*
Verify Corrected Design		*HLSFixer retests the corrected HLS design against the golden results to ensure semantic equivalence P29*

Verify objective completion	Independently check whether a goal state has truly been achieved before advancing — guards against false-positive task completion where the agent believes it succeeded but the environment has not changed.	*"Independently checks whether objectives are truly complete, preventing the orchestrator from advancing when the main agent incorrectly believes a task is finished" (P25)*
Formulate Repair Instructions		*"This model formulates explicit modification instructions with detailed analysis" (P29)*
Analyse log to formulate error modification actions		*an analysis agent adopts a reasoning-to-instruction method, analyzing the HLS log to formulate error modification actions" (P29)*
Assess Bug Applicability		*The agent assesses the contextual applicability of potential bugs, reducing the probability that the LLM forcibly generates trivial results. (P29)*
Review Code Against Golden Standard		*"we prompt LLM to review the correct HLS-C code, pairing buggy code segments with corresponding error messages to construct debugging CoT" P29*
Refine Directive Strategy Iteratively		*When the initial attempt fails to meet the desired performance, HLSTuner activates an iterative refinement incorporating current directives and the resulting QoR." P29* *"The results of the execution are fed back, enabling GPT-4 to refine its responses and propose further iterations" P32* *"This iterative cycle continues until GPT-4 determines that the accumulated information suffices to conclusively answer the problem" P32*

- Decision-Making Actions

Action	Plain Explanation	
Propose action candidates	Brainstorm "what could I do right now?" — generate one or more possible next moves.	
Revise output based on evaluator feedback		*In subsequent steps, feedback from the feedback agent is used to iteratively refine and improve the generated themes. P27*
Simulate grounding feedback internally	Mentally rehearse an action before doing it — imagine "if I do X, what will the environment look like?" without actually executing.	
Select actions via argmax, softmax, or majority vote	Pick one action to execute: take the highest-scored one (argmax), sample probabilistically (softmax), or let multiple evaluators vote.	
Execute grounding actions, API calls, or learning actions	Finally, do it for real: either perform an external action (call an API, move a robot) or an internal one (update memory, fine-tune).	
	Insert Directives into Code	*"an insertion agent executes this plan for HLS-C optimization" P29*
Aggregate multiple candidate outputs		*aggregates prediction results from multiple tools to generate the final cell type labels P1*

~~Anonymize candidates before evaluation~~		~~*all algorithm identifiers are masked before evaluation … replaced by generic labels P1*~~
Generate candidate pipelines for comparison		*Running multiple candidate pipelines for each analysis step P1*
Generate Candidate Repairs		*"LLM Group for multifaceted evaluation generates diverse debugging instructions" (P29)*
Execute Code Repair		*“Operating under strict instruction adherence, this agent adopts the instructions to implement debugging.” (P29)*

• Boundary-Aware Actions

Actions where the agent detects, communicates, or manages the limits of its knowledge, role, or capabilities. These actions concern the agent's epistemic self-model and refusal behavior. Distinct from Decision-Making (action selection) and Communication (general output).

Action (Verb + Noun)	Plain Explanation	Evidence
Recognize Knowledge Boundary	Determine whether an incoming query falls outside the agent's role knowledge or capability scope — the prerequisite for any refusal action.	*" recognize and refuse queries that conflict with their role knowledge" (P22)*
Recognize Cognition Boundary	Respond to the query with proactive identity disclosure to bound	*“identity disclosure, S2S systems often proactively mention that they are intelligent assistants, which*

Action (Verb + Noun)	Plain Explanation	Evidence
		undermine the premise of Turing test" (P11)
Refuse Query	Actively decline to answer a query that exceeds the agent's knowledge boundary or role setting, signaling that the question is out of scope.	*"appropriately reject queries that exceed their knowledge boundaries or conflict with their role settings" (P22)*
Generate Refusal Response	Produce a character-consistent refusal message with an appropriate explanation of why the query cannot be answered.	*"providing clear refusal responses with appropriate explanations" (P22)*
Acknowledge False Information	Identify and surface false premises embedded in an incoming query, signaling to the user what is incorrect before or instead of answering.	*"Evaluating whether the model recognizes potential errors in the query" (P22)*

• Role Conditioning Actions 🆕 (new category)

Actions by which agents take on, maintain, or leverage domain-expert personas to shape how they process and generate output at inference time. These actions change the framing lens of the agent without modifying model parameters.

Action (Verb + Noun)	Plain Explanation	Evidence
🆕 **Adopt Expert Persona**	Role-condition the agent with a domain identity (e.g., Physician, Cardiac Surgeon, Qualitative Researcher) at prompt	*"A pool of k=4 role-conditioned GPT-4o P27*

Action (Verb + Noun)	Plain Explanation	Evidence
	invocation time to bias its analytical perspective toward domain-specific patterns.	*agents are each given the full interview transcript as input" P27*
🆕 **Retain Role-Specific Perspective**	Maintain a domain-expert lens through downstream processing steps to produce perspective-diverse outputs alongside identity-neutral agents.	*"others may retain role-specific perspectives to support diverse theme formulation" P27*

V2 (validated):

Verb + Noun format · with plain-English explanations

Differences between Learning actions, reasoning actions, and decision-making actions.

- Memory Actions

Action	Plain Explanation
Store information in working memory	Put whatever the agent is currently using — recent inputs, intermediate results — onto a "scratch pad" for quick access.